

Technical Documentation

1. Introduction

What is RAG? Retrieval-Augmented Generation (RAG) is an architectural framework that improves the efficacy of Large Language Models (LLMs) by grounding them in external, domain-specific data before generating a response.

Why it is needed: Base LLMs suffer from "hallucinations" and lack access to private or up-to-date information. RAG solves this by injecting factual, retrieved context directly into the LLM's prompt.

Use Case Overview: This system serves as a highly reliable Customer Support Assistant. It ingests static policy documents (PDFs) and answers user queries autonomously, escalating to a human agent only when it lacks the confidence or context to proceed safely.

2. System Architecture Explanation

The system is split into an offline ingestion pipeline and a real-time query orchestrator. The ingestion phase uses LangChain to extract PDF text, chunk it, embed it via an OpenAI embedding model, and persist it in a ChromaDB vector database. The real-time phase uses LangGraph to orchestrate a state machine. When a user queries the system, LangGraph routes the input to a retrieval node, fetches vector matches, and passes them to an LLM. The LLM evaluates the context and outputs an answer alongside a confidence metric, which LangGraph uses to determine if a human agent must intervene.

3. Design Decisions

- **Chunk Size Choice:** We selected a chunk size of 500 tokens with a 50-token overlap. This strikes a balance: it is small enough to ensure semantic density (retrieving highly specific answers) but large enough to provide sufficient context to the LLM without fragmenting sentences.
- **Embedding Strategy:** OpenAI's `text-embedding-3-small` was chosen for its high dimensionality (1536) and cost-to-performance ratio, outperforming older open-source models in semantic clustering.
- **Retrieval Approach:** We utilize Cosine Similarity to fetch the top-3 most relevant chunks ($k=3$). This ensures the context window remains focused and reduces LLM token costs.
- **Prompt Design Logic:** The system prompt strictly enforces constraints: *"You are a support agent. Answer ONLY using the provided context. If the context does not contain the*

answer, output a confidence score of 0.0. Output a JSON object containing your 'draft_answer' and 'confidence_score'."

4. Workflow Explanation (LangGraph)

LangGraph manages the system's state using a cyclical graph structure.

- **Node Responsibilities:**

1. **Retrieve:** Fetches data from ChromaDB.
2. **Evaluate:** Calls the LLM to draft an answer and score it.
3. **HITL:** Pauses execution to wait for a human.
4. **Output:** Formats the final response to the user.

- **State Transitions:** The `AgentState` dictionary is passed between nodes. The graph transitions linearly from Retrieval to Evaluation, but uses a conditional edge after Evaluation to route to either Output or HITL.

5. Conditional Logic & Routing

The system's decision-making relies on strict intent detection and routing decisions. The LLM acts as an evaluator. If the LLM's self-reported confidence score is ≥ 0.85 AND the retrieved context is non-empty, the system routes to the Output node. If the score is < 0.85 , context is missing, or the query requires account modification (complex intent), the conditional edge actively routes the state to the HITL node.

6. HITL Implementation

Role of Human Intervention: The Human-in-the-Loop serves as a safety net to guarantee zero hallucinations on edge-case queries. LangGraph uses a `NodeInterrupt` to suspend the workflow.

Benefits & Limitations: Benefits include 100% accuracy on complex tickets and continuous user trust. The primary limitation is the introduction of asynchronous latency; the user must wait for a human to respond, breaking the illusion of an instant chatbot.

7. Challenges & Trade-offs

- **Retrieval Accuracy vs. Speed:** Fetching more chunks (e.g., $k=10$) increases the likelihood of finding the right answer but significantly slows down the LLM generation time and increases token costs. We settled on $k=3$ to prioritize speed while maintaining acceptable accuracy.

- **Chunk Size vs. Context Quality:** Larger chunks provide better narrative context but can dilute the semantic matching in the vector database. We chose smaller, 500-token chunks with overlap to prevent cutting off critical sentences.
- **Cost vs. Performance:** We utilized `gpt-4o-mini` as the LLM engine. While `gpt-4o` yields slightly better reasoning, the "mini" variant reduces operational costs by roughly 90% while still expertly handling standard support queries.

8. Testing Strategy

The system is tested using a two-pronged approach: Unit tests for individual node logic, and Integration tests for the LangGraph routing edges.

Sample Queries:

- *Standard:* "How do I update my billing address?" (Expected: Handled by Bot).
- *Escalation:* "My account was double-charged and I want a refund to a different card." (Expected: Low confidence, routed to HITL).

9. Future Enhancements

- **Multi-Document Support:** Upgrading the metadata schema in ChromaDB to filter queries by document type (e.g., routing billing queries to the billing PDF, and technical queries to the API PDF).
- **Memory Integration:** Implementing `ConversationBufferMemory` to allow users to ask follow-up questions without losing context of previous turns.
- **Feedback Loop:** Adding a thumbs-up/down UI element to the bot's responses. Failed responses will be logged and used to fine-tune the chunking strategy or prompt design.